

WhatsApp-First School ERP

Technical Foundation and Database Architecture

System Architecture Specification

September 16, 2026

Abstract

This document defines the technical foundation for a multi-school Enterprise Resource Planning (ERP) platform designed specifically for schools. The system uses WhatsApp as a primary operational interface for routine actions and a React-based web application for complex administrative workflows. A Rust backend provides the central API and business-logic layer, PostgreSQL is the system of record, Cloudflare R2 is intended for object storage, and the MVP is planned for deployment on Render before migration to larger hosting infrastructure as required. An AI layer is planned for natural-language command interpretation, initially using an NVIDIA API.

The most important architectural principle is that WhatsApp and the web application are interfaces to the same ERP core. Business rules, authorization, validation, database operations, audit logging, and transactions remain inside the Rust backend. AI may interpret a message into a structured command, but it must not directly modify the database.

Contents

1	Product Overview	5
2	Technology Stack	5
3	Core Architecture	6
4	Architectural Principles	7
4.1	Single Business Logic Layer	7
4.2	AI Is Not the Database Authority	7
4.3	Multi-Tenant School Isolation	8
4.4	Academic-Year Awareness	8
5	Domain Model	8
6	Database Design	8
6.1	School and Tenancy	8
6.1.1	schools	8
6.1.2	school_branches	9
6.1.3	school_settings	9
7	Academic Calendar	10
7.1	academic_years	10
7.2	academic_terms	10
7.3	academic_calendar	10

8 Identity and Access Management	10
8.1 users	10
8.2 roles	11
8.3 permissions	11
8.4 user_roles	12
8.5 role_permissions	12
8.6 user_sessions	12
8.7 email_verifications and password_resets	13
9 People	13
9.1 persons	13
9.2 staff	13
9.3 departments	14
9.4 staff_user_accounts	14
10 Students and Guardians	14
10.1 students	14
10.2 guardians	14
10.3 student_guardians	15
10.4 student_addresses	15
10.5 student_documents	15
11 Admissions	15
11.1 admissions	16
11.2 admission_applicants	16
11.3 admission_guardians	16
11.4 custom_fields	17
11.5 admission_custom_values	17
11.6 admission_status_history	17
12 Enrollment and Academic Structure	17
12.1 enrollments	17
12.2 classes	18
12.3 sections	18
12.4 class_sections	18
12.5 subjects	19
12.6 academic_subjects	19
12.7 class_subjects	19
12.8 teaching_groups	19
12.9 teaching_group_members	19
12.10rooms	20
13 Attendance	20
13.1 attendance_sessions	20
13.2 attendance_records	20
13.3 attendance_corrections	21
14 Timetable	21
14.1 periods	21
14.2 timetable_versions	22
14.3 timetable_entries	22
14.4 substitutions	22

14.5 teacher_availability	23
15 Examinations and Results	23
15.1 exam_groups	23
15.2 exams	23
15.3 exam_subjects	23
15.4 student_marks	24
15.5 grading_scales and grading_scale_items	24
15.6 report_cards	24
16 Fees and Finance	24
16.1 fee_categories	25
16.2 fee_structures	25
16.3 fee_structure_items	25
16.4 student_fee_assignments	25
16.5 fee_invoices	25
16.6 fee_invoice_items	26
16.7 fee_payments	26
17 WhatsApp Architecture	26
17.1 whatsapp_accounts	26
17.2 whatsapp_contacts	27
17.3 whatsapp_conversations	27
17.4 whatsapp_messages	27
17.5 whatsapp_verifications	27
18 WhatsApp Command Processing	28
18.1 whatsapp_commands	28
19 AI Layer	29
19.1 ai_requests	29
19.2 ai_command_executions	29
20 Notifications and Email	29
20.1 notification_templates	29
20.2 notifications	30
20.3 email_messages	30
21 File Storage	30
21.1 files	30
22 Audit Logging	31
22.1 audit_logs	31
23 Webhooks and Background Jobs	31
23.1 webhook_events	31
23.2 jobs	32
24 Integrations	32
24.1 integrations	32
25 Core Entity Relationships	33

26 MVP Database Scope	34
26.1 MVP Foundation	34
26.2 MVP Academic Layer	34
26.3 MVP People Layer	34
26.4 MVP Admission Layer	35
26.5 MVP Attendance Layer	35
26.6 MVP WhatsApp Layer	35
26.7 MVP AI and Infrastructure	35
27 Recommended Development Order	35
28 Rust Backend Architecture	36
29 Deployment Architecture	37
30 Security Requirements	38
31 Important Design Decisions	38
31.1 Student vs Admission	38
31.2 Student vs Enrollment	39
31.3 AI vs Business Logic	39
31.4 WhatsApp vs Web	39
32 Initial WhatsApp Command Set	39
33 Future Modules	39
34 Conclusion	40

1 Product Overview

The ERP is a school management platform with two complementary interfaces:

- **WhatsApp interface:** optimized for fast, repetitive, operational activities.
- **Web interface:** optimized for complex, visual, administrative, and configuration-heavy workflows.

The core product idea is:

Do not force every school employee to learn a complex ERP interface. Let authorized users operate routine ERP functions through WhatsApp while providing a complete web interface for advanced workflows.

Examples of WhatsApp-oriented actions include:

- Taking attendance.
- Marking individual students absent or present.
- Correcting attendance.
- Performing approved bulk operations.
- Looking up student information.
- Requesting attendance reports.
- Querying school information.
- Sending or requesting routine notifications.

Examples of web-oriented workflows include:

- Timetable construction.
- Class and academic configuration.
- Admission review and approval.
- Complex reports.
- Fee configuration.
- Examination configuration.
- User and permission administration.
- Bulk imports and large data operations.

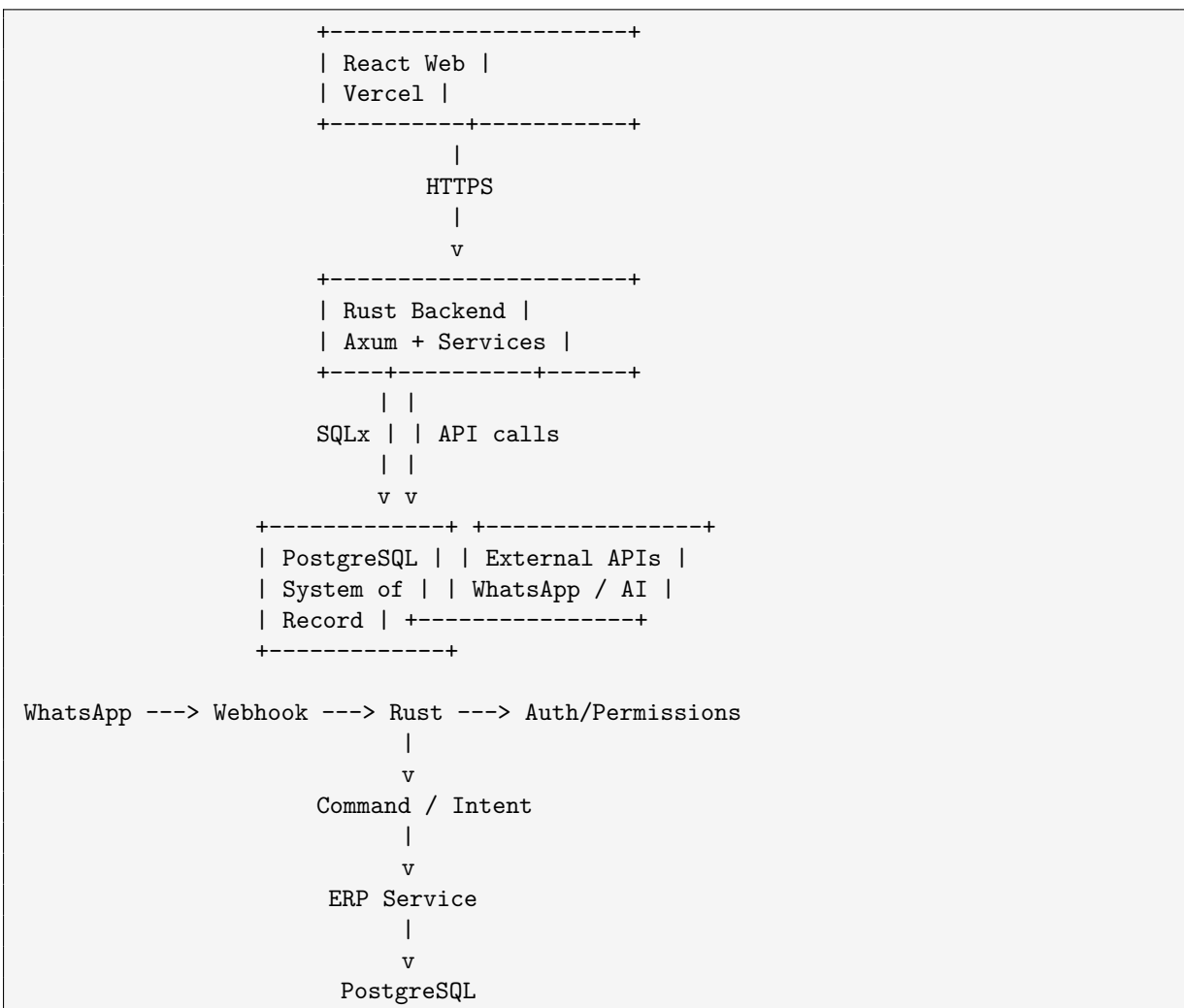
The web interface should still expose routine functions where useful. WhatsApp is a convenience and operational interface, not a restriction.

2 Technology Stack

Component	Technology / Purpose
Frontend	React
Frontend hosting	Vercel
Backend	Rust
Web framework	Axum (planned)
Async runtime	Tokio (planned)
Database access	SQLx (planned)
Database	PostgreSQL
MVP backend hosting	Render
Object storage	Cloudflare R2
Messaging interface	WhatsApp Business API
AI provider	NVIDIA API initially
Containerization	Docker
Authentication	Rust backend with session/JWT strategy as finalized

3 Core Architecture

The system follows a central ERP-core architecture.



The same ERP service must be reusable by both web and WhatsApp interfaces:

React Web -----\

```

                > Rust API -> ERP Service -> PostgreSQL
WhatsApp -----/

```

This avoids implementing attendance, admissions, permissions, or other business logic twice.

4 Architectural Principles

4.1 Single Business Logic Layer

Business rules must reside in Rust services rather than inside React or WhatsApp handlers.

4.2 AI Is Not the Database Authority

The AI layer should transform natural language into a structured command. Rust must then:

1. Authenticate the user.
2. Identify the school and operational context.
3. Check permissions.
4. Validate the command.
5. Resolve entities such as students and classes.
6. Execute the appropriate ERP service.
7. Commit the database transaction where appropriate.
8. Record an audit event.
9. Return a controlled response.

The desired flow is:

```

WhatsApp message
  |
  v
AI intent extraction
  |
  v
Structured command
  |
  v
Rust validation
  |
  v
RBAC / permission check
  |
  v
ERP service
  |
  v
PostgreSQL transaction
  |
  v
Audit log
  |
  v
WhatsApp response

```

4.3 Multi-Tenant School Isolation

The database is designed around schools as tenants. School-owned entities should carry a school relationship directly or through a clearly constrained parent entity.

Every request must establish the school context before accessing school data.

4.4 Academic-Year Awareness

School data changes over time. Students remain permanent entities, while their enrollment, class, section, subjects, and timetable assignments can change between academic years.

5 Domain Model

The major domains are:

1. School tenancy and configuration.
2. Identity, authentication, roles, and permissions.
3. People, staff, teachers, students, and guardians.
4. Admissions.
5. Academic years and calendars.
6. Classes, sections, subjects, and teaching groups.
7. Student enrollment.
8. Attendance.
9. Timetable and substitutions.
10. Examinations and grading.
11. Fees and finance.
12. WhatsApp.
13. AI command processing.
14. Notifications and email.
15. Files and object storage.
16. Audit and system operations.

6 Database Design

6.1 School and Tenancy

6.1.1 schools

Top-level school tenant.

```
schools
-----
id UUID PRIMARY KEY
name VARCHAR
code VARCHAR UNIQUE
short_name VARCHAR
email VARCHAR
phone VARCHAR
website VARCHAR
address_line_1 TEXT
address_line_2 TEXT
city VARCHAR
district VARCHAR
state VARCHAR
country VARCHAR
postal_code VARCHAR
timezone VARCHAR
logo_file_id UUID
status ENUM
created_at TIMESTAMP
updated_at TIMESTAMP
```

6.1.2 school_branches

Supports multiple campuses.

```
school_branches
-----
id
school_id FK schools
name
code
address
phone
email
status
created_at
updated_at
```

6.1.3 school_settings

Stores configurable school settings. Frequently used and structurally important settings may later be promoted to dedicated columns.

```
school_settings
-----
id
school_id
key
value
created_at
updated_at
```

7 Academic Calendar

7.1 academic_years

```
academic_years
-----
id
school_id
name
start_date
end_date
status
created_at
updated_at

UNIQUE(school_id, name)
```

Example: 2026-27.

7.2 academic_terms

```
academic_terms
-----
id
academic_year_id
name
code
start_date
end_date
sequence
status
```

7.3 academic_calendar

Represents holidays, working days, examinations, events, and special dates.

```
academic_calendar
-----
id
school_id
academic_year_id
date
type
title
description
is_working_day
created_at
updated_at
```

8 Identity and Access Management

8.1 users

A user represents an account capable of authenticating into the ERP.

```
users
-----
id
school_id
branch_id
username
email
phone
password_hash
first_name
last_name
display_name
status
last_login_at
created_at
updated_at
```

8.2 roles

```
roles
-----
id
school_id
name
code
description
is_system_role
created_at
updated_at
```

Initial examples:

- SUPER_ADMIN
- SCHOOL_ADMIN
- PRINCIPAL
- TEACHER
- ACCOUNTANT
- RECEPTIONIST
- STAFF

8.3 permissions

```
permissions
-----
id
code
name
description
```

Example permission codes:

```
student.read
student.create
student.update
student.delete

attendance.read
attendance.create
attendance.update

admission.read
admission.approve

timetable.read
timetable.create
timetable.update

fees.read
fees.create
fees.refund
```

8.4 user_roles

```
user_roles
-----
id
user_id
role_id
created_at
```

8.5 role_permissions

```
role_permissions
-----
role_id
permission_id

PRIMARY KEY(role_id, permission_id)
```

8.6 user_sessions

```
user_sessions
-----
id
user_id
token_hash
ip_address
user_agent
expires_at
created_at
revoked_at
```

8.7 email_verifications and password_resets

These support account verification and password recovery.

```
email_verifications
-----
id
user_id
token_hash
expires_at
verified_at
created_at

password_resets
-----
id
user_id
token_hash
expires_at
used_at
created_at
```

9 People

9.1 persons

A reusable human identity.

```
persons
-----
id
first_name
middle_name
last_name
display_name
date_of_birth
gender
phone
email
photo_file_id
created_at
updated_at
```

9.2 staff

```
staff
-----
id
school_id
branch_id
person_id
employee_code
joining_date
employment_type
designation
department_id
```

```
status
created_at
updated_at
```

9.3 departments

```
departments
-----
id
school_id
name
code
description
```

9.4 staff_user_accounts

Connects staff members to optional ERP accounts.

```
staff_user_accounts
-----
staff_id
user_id
```

10 Students and Guardians

10.1 students

A permanent student identity.

```
students
-----
id
school_id
admission_number
student_code
person_id
date_of_admission
status
created_at
updated_at
```

A student's class should not be stored directly on this table because class membership changes by academic year.

10.2 guardians

```
guardians
-----
id
person_id
occupation
created_at
updated_at
```

10.3 student_guardians

Many-to-many relationship between students and guardians.

```
student_guardians
-----
id
student_id
guardian_id
relationship
is_primary
is_emergency_contact
can_receive_notifications
created_at
```

10.4 student_addresses

```
student_addresses
-----
id
student_id
type
address_line_1
address_line_2
city
district
state
postal_code
country
is_primary
```

10.5 student_documents

```
student_documents
-----
id
student_id
document_type
file_id
document_number
issued_date
expiry_date
verified
verified_by
verified_at
```

11 Admissions

Admissions are deliberately separated from students.

The lifecycle is:

```
ADMISSION
|
v
APPROVED
```

```

|
v
STUDENT
|
v
ENROLLMENT

```

11.1 admissions

```

admissions
-----
id
school_id
academic_year_id
application_number
status
submitted_at
approved_at
approved_by
rejected_at
rejected_by
rejection_reason
created_at
updated_at

```

Suggested statuses:

```

DRAFT
SUBMITTED
UNDER_REVIEW
APPROVED
REJECTED
CANCELLED
CONVERTED

```

11.2 admission_applicants

```

admission_applicants
-----
id
admission_id
first_name
middle_name
last_name
date_of_birth
gender
phone
email

```

11.3 admission_guardians

```

admission_guardians
-----
id
admission_id

```

```
name
relationship
phone
email
occupation
```

11.4 custom_fields

Supports school-specific fields without changing the database schema.

```
custom_fields
-----
id
school_id
entity_type
field_name
field_key
field_type
is_required
options_json
display_order
created_at
updated_at
```

11.5 admission_custom_values

```
admission_custom_values
-----
id
admission_id
field_id
value
```

11.6 admission_status_history

```
admission_status_history
-----
id
admission_id
from_status
to_status
changed_by
reason
created_at
```

12 Enrollment and Academic Structure

12.1 enrollments

Enrollment connects a student to an academic year and class/section.

```
enrollments
-----
id
```

```
student_id
academic_year_id
class_id
section_id
roll_number
admission_id
enrollment_date
status
created_at
updated_at
```

Example:

```
Student Rahul
|
+-- 2026-27 -> Class 8 -> Section A
|
+-- 2027-28 -> Class 9 -> Section B
```

12.2 classes

```
classes
-----
id
school_id
name
code
sequence
```

12.3 sections

```
sections
-----
id
school_id
name
code
```

12.4 class_sections

Represents a class/section grouping for a particular academic year.

```
class_sections
-----
id
academic_year_id
class_id
section_id
class_teacher_staff_id
capacity
room_id
status
```

12.5 subjects

```
subjects
-----
id
school_id
name
code
subject_type
description
status
```

12.6 academic_subjects

```
academic_subjects
-----
id
academic_year_id
subject_id
```

12.7 class_subjects

```
class_subjects
-----
id
class_section_id
academic_subject_id
```

12.8 teaching_groups

Supports subject groups, electives, labs, and other cases where a whole section is not taught as one group.

```
teaching_groups
-----
id
class_section_id
name
code
type
```

12.9 teaching_group_members

```
teaching_group_members
-----
id
teaching_group_id
student_id
```

12.10 rooms

```
rooms
-----
id
school_id
branch_id
name
code
capacity
room_type
```

13 Attendance

Attendance is a core WhatsApp operation.

13.1 attendance_sessions

Represents a class/date/period attendance event.

```
attendance_sessions
-----
id
school_id
academic_year_id
class_section_id
teaching_group_id
date
period_id
attendance_type
marked_by
status
created_at
updated_at
```

13.2 attendance_records

```
attendance_records
-----
id
attendance_session_id
student_id
status
marked_at
marked_by
remarks
```

Suggested statuses:

```
PRESENT
ABSENT
LATE
EXCUSED
```

13.3 attendance_corrections

Attendance corrections should not silently erase the original state.

```
attendance_corrections
-----
id
attendance_record_id
old_status
new_status
reason
changed_by
created_at
```

Example WhatsApp flow:

```
Teacher:
"Mark Rahul and Arjun absent in 8A"

    |
    v

AI:
MARK_ATTENDANCE

    |
    v

Rust:
- identify school
- identify teacher
- resolve 8A
- resolve students
- check permission
- validate date/session
- execute transaction

    |
    v

PostgreSQL

    |
    v

WhatsApp:
"Rahul and Arjun have been marked absent."
```

14 Timetable

Timetable construction is primarily a web-interface workflow.

14.1 periods

```
periods
-----
id
```

```
school_id
name
sequence
start_time
end_time
```

14.2 timetable_versions

Allows timetable changes and effective dates.

```
timetable_versions
-----
id
school_id
academic_year_id
name
effective_from
effective_to
status
created_by
created_at
```

14.3 timetable_entries

The design explicitly supports day, period, subject, teaching group, and custom entries.

```
timetable_entries
-----
id
timetable_version_id
class_section_id
day_of_week
period_id
subject_id
teaching_group_id
teacher_id
room_id
entry_type
custom_label
```

Suggested entry types:

```
NORMAL
CUSTOM
EVENT
FREE
```

14.4 substitutions

```
substitutions
-----
id
school_id
date
period_id
class_section_id
```

```
original_teacher_id
substitute_teacher_id
subject_id
room_id
reason
status
created_by
created_at
```

14.5 teacher_availability

```
teacher_availability
-----
id
staff_id
day_of_week
period_id
status
```

15 Examinations and Results

15.1 exam_groups

```
exam_groups
-----
id
school_id
academic_year_id
name
term_id
start_date
end_date
status
```

15.2 exams

```
exams
-----
id
exam_group_id
name
```

15.3 exam_subjects

```
exam_subjects
-----
id
exam_id
subject_id
exam_date
start_time
end_time
```

```
max_marks
pass_marks
```

15.4 student_marks

```
student_marks
-----
id
exam_subject_id
student_id
marks
grade
remarks
entered_by
updated_by
```

15.5 grading_scales and grading_scale_items

```
grading_scales
-----
id
school_id
name

grading_scale_items
-----
id
grading_scale_id
grade
min_percentage
max_percentage
description
```

15.6 report_cards

```
report_cards
-----
id
student_id
academic_year_id
term_id
status
generated_file_id
generated_at
```

16 Fees and Finance

The finance system should be modular. The MVP can begin with fees and payments before introducing a full accounting subsystem.

16.1 fee_categories

```
fee_categories
-----
id
school_id
name
code
```

16.2 fee_structures

```
fee_structures
-----
id
school_id
academic_year_id
name
```

16.3 fee_structure_items

```
fee_structure_items
-----
id
fee_structure_id
fee_category_id
amount
frequency
due_day
```

16.4 student_fee_assignments

```
student_fee_assignments
-----
id
student_id
fee_structure_id
academic_year_id
```

16.5 fee_invoices

```
fee_invoices
-----
id
student_id
academic_year_id
invoice_number
invoice_date
due_date
total_amount
status
```

16.6 fee_invoice_items

```
fee_invoice_items
-----
id
invoice_id
fee_category_id
description
amount
```

16.7 fee_payments

```
fee_payments
-----
id
invoice_id
student_id
amount
payment_method
transaction_reference
paid_at
received_by
status
```

Future finance tables may include:

- accounts
- account_transactions
- expense_categories
- expenses
- refunds

17 WhatsApp Architecture

17.1 whatsapp_accounts

Stores school-level WhatsApp Business configuration. Sensitive credentials must be encrypted or stored through a secure secrets mechanism.

```
whatsapp_accounts
-----
id
school_id
phone_number
phone_number_id
business_account_id
access_token_encrypted
status
created_at
updated_at
```

17.2 whatsapp_contacts

```
whatsapp_contacts
-----
id
school_id
phone_number
name
user_id
person_id
student_id
guardian_id
status
created_at
updated_at
```

17.3 whatsapp_conversations

```
whatsapp_conversations
-----
id
school_id
contact_id
status
started_at
last_message_at
```

17.4 whatsapp_messages

```
whatsapp_messages
-----
id
conversation_id
message_id
direction
message_type
content
media_file_id
status
sent_at
received_at
```

17.5 whatsapp_verifications

```
whatsapp_verifications
-----
id
contact_id
otp_hash
attempts
expires_at
verified_at
created_at
```

The intended identity chain is:

```

WhatsApp Number
  |
  v
Verified Contact
  |
  v
Person
  |
  v
User
  |
  v
School
  |
  v
Roles / Permissions

```

18 WhatsApp Command Processing

18.1 whatsapp_commands

Raw messages and executable commands should be separate concepts.

```

whatsapp_commands
-----
id
message_id
user_id
command_type
raw_text
parsed_payload
status
requires_confirmation
confirmed_at
executed_at
error_message
created_at

```

Example:

```

raw_text:
"Mark Rahul absent in 8A"

command_type:
MARK_ATTENDANCE

parsed_payload:
{
  "class": "8A",
  "student": "Rahul",
  "status": "ABSENT"
}

```

Commands that have destructive, high-impact, or ambiguous consequences should be capable of requiring confirmation.

19 AI Layer

The AI provider is an implementation detail. The database should not be designed specifically around NVIDIA so that another provider or a local model can be introduced later.

19.1 ai_requests

```
ai_requests
-----
id
school_id
user_id
channel
provider
model
input_text
output_json
intent
tokens_used
status
created_at
```

19.2 ai_command_executions

```
ai_command_executions
-----
id
ai_request_id
command_type
payload
validation_status
permission_status
execution_status
executed_by
executed_at
error
```

AI should be treated as an interpreter, not an authorization system.

20 Notifications and Email

20.1 notification_templates

```
notification_templates
-----
id
school_id
channel
event_type
template_name
content
status
```

20.2 notifications

```
notifications
-----
id
school_id
user_id
channel
template_id
recipient
payload
status
sent_at
created_at
```

20.3 email_messages

```
email_messages
-----
id
school_id
recipient
subject
body
template_id
status
provider_message_id
sent_at
created_at
```

21 File Storage

Files should normally be stored in Cloudflare R2 rather than directly inside PostgreSQL.

21.1 files

```
files
-----
id
school_id
storage_provider
bucket
object_key
original_name
mime_type
size_bytes
checksum
uploaded_by
created_at
```

The relationship is:

```
PostgreSQL
|
+-- file metadata / object_key
```

|
v
Cloudflare R2

22 Audit Logging

A serious ERP requires traceability.

22.1 audit_logs

```
audit_logs
-----
id
school_id
user_id
action
entity_type
entity_id
old_values
new_values
ip_address
user_agent
source
created_at
```

Suggested sources:

```
WEB
WHATSAPP
API
SYSTEM
AI
```

Examples of audit questions the system should answer:

- Who changed a student's attendance?
- What was the previous value?
- What was the new value?
- Was the change made through WhatsApp or the web?
- Which authenticated user performed it?

23 Webhooks and Background Jobs

23.1 webhook_events

```
webhook_events
-----
id
provider
event_type
external_event_id
payload
```

```
status
received_at
processed_at
```

This supports idempotent webhook processing and debugging.

23.2 jobs

```
jobs
----
id
school_id
job_type
payload
status
scheduled_at
started_at
completed_at
error
attempts
```

Potential jobs include:

- scheduled notifications
- report generation
- email delivery
- WhatsApp message processing
- reminders
- document processing

24 Integrations

24.1 integrations

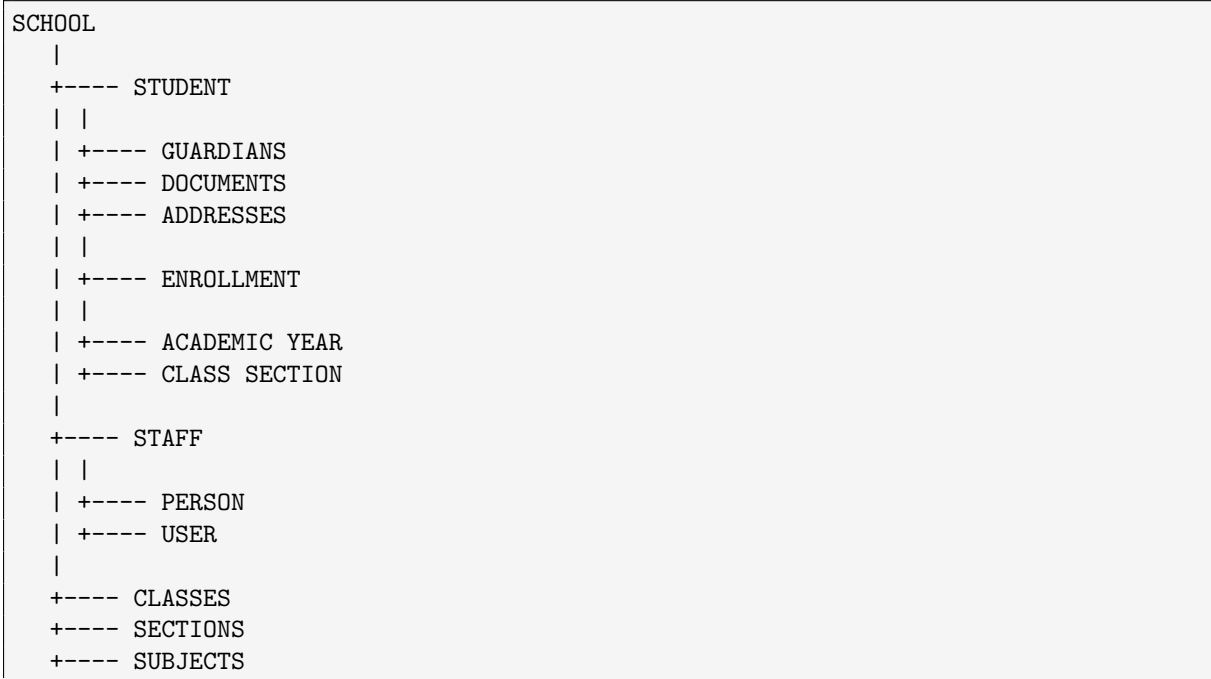
External integrations should be abstracted rather than hard-coded into business logic.

```
integrations
-----
id
school_id
provider
type
credentials_encrypted
status
created_at
updated_at
```

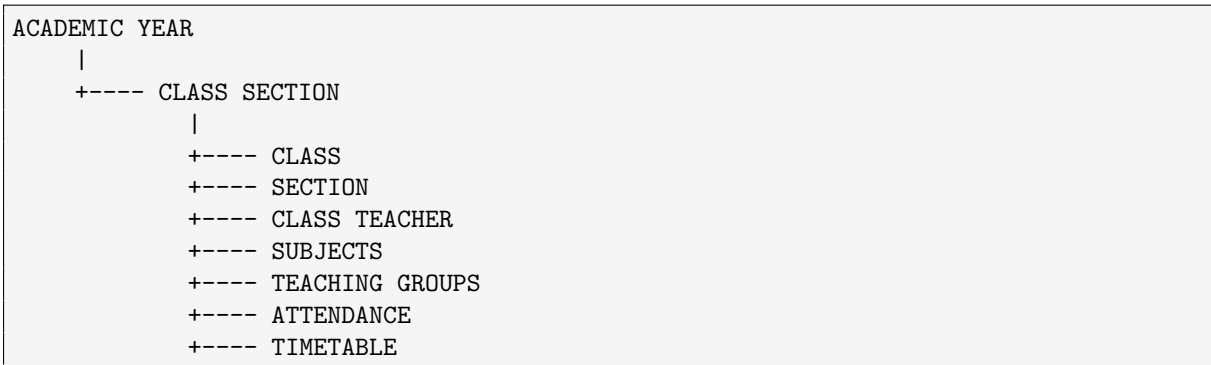
Possible providers include WhatsApp, payment gateways, email, AI providers, Google Drive, and Cloudflare.

25 Core Entity Relationships

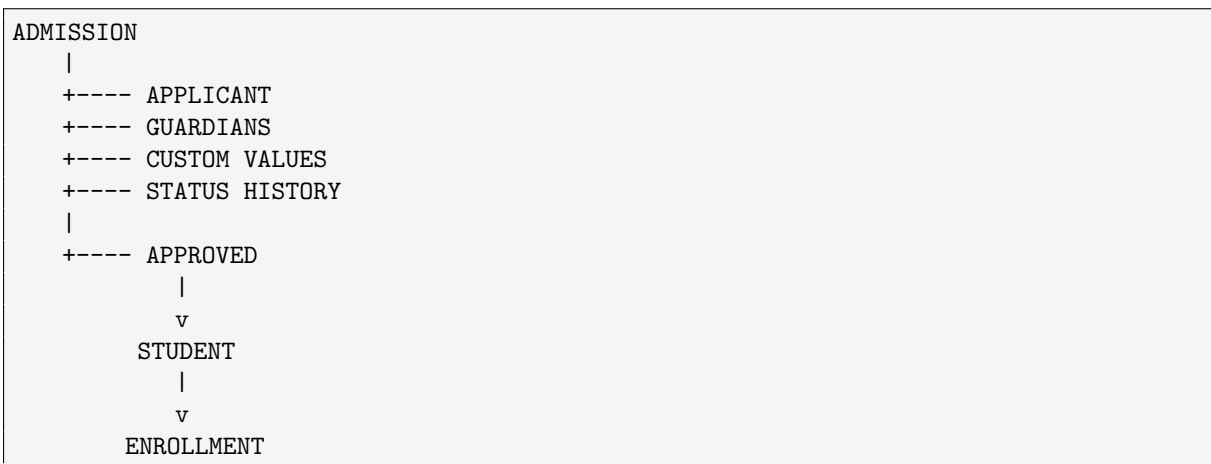
The primary student relationship is:



Academic structure:



Admission structure:



26 MVP Database Scope

The full architecture can eventually contain many modules, but the MVP should be developed incrementally.

26.1 MVP Foundation

- schools
- school_branches
- school_settings
- users
- roles
- permissions
- user_roles
- role_permissions
- user_sessions

26.2 MVP Academic Layer

- academic_years
- academic_terms
- academic_calendar
- classes
- sections
- class_sections
- subjects
- academic_subjects
- class_subjects
- periods
- rooms

26.3 MVP People Layer

- persons
- staff
- students
- guardians
- student_guardians
- enrollments

26.4 MVP Admission Layer

- admissions
- admission_applicants
- admission_guardians
- custom_fields
- admission_custom_values
- admission_status_history

26.5 MVP Attendance Layer

- attendance_sessions
- attendance_records
- attendance_corrections

26.6 MVP WhatsApp Layer

- whatsapp_accounts
- whatsapp_contacts
- whatsapp_conversations
- whatsapp_messages
- whatsapp_verifications
- whatsapp_commands

26.7 MVP AI and Infrastructure

- ai_requests
- ai_command_executions
- files
- audit_logs
- webhook_events
- jobs

27 Recommended Development Order

The recommended implementation order is:

1. Rust project and application configuration.
2. PostgreSQL connection and migrations.
3. Schools and tenant isolation.

4. Users, authentication, roles, and permissions.
5. Academic years.
6. Classes, sections, and class sections.
7. People, staff, students, and guardians.
8. Enrollments.
9. Subjects.
10. Attendance.
11. Basic Rust CRUD APIs.
12. React web administration interface.
13. WhatsApp authentication.
14. WhatsApp webhook processing.
15. WhatsApp command infrastructure.
16. AI intent extraction.
17. Permission and validation pipeline.
18. Attendance through WhatsApp.
19. Admissions.
20. Timetable and substitutions.
21. Examinations.
22. Fees and finance.
23. Reports and advanced administration.

28 Rust Backend Architecture

The Rust backend should be organized around modules/domains rather than a single large file.
A suggested structure is:

```
src/  
  main.rs  
  config.rs  
  db.rs  
  error.rs  
  
  middleware/  
    auth.rs  
    permissions.rs  
  
  modules/  
    schools/  
    users/  
    students/  
    admissions/
```

```

academics/
attendance/
timetable/
exams/
fees/
whatsapp/
ai/
notifications/

services/

```

A typical operation should follow:

```

HTTP / WhatsApp request
|
v
Handler
|
v
Authentication
|
v
Authorization
|
v
Validation
|
v
Service
|
v
Repository / SQLx
|
v
PostgreSQL
|
v
Audit

```

29 Deployment Architecture

Initial MVP:

```

React
|
+--> Vercel

Rust API
|
+--> Docker
    |
    +--> Render

PostgreSQL
|
+--> Managed PostgreSQL

Files

```

```

|
+--> Cloudflare R2

WhatsApp
|
+--> WhatsApp Business API
      |
      +--> Rust webhook

AI
|
+--> NVIDIA API

```

The deployment architecture should remain portable so that the Rust backend can later move from Render to another hosting platform without changing the application architecture.

30 Security Requirements

Security must be treated as a core ERP feature.

- Passwords must be stored using secure password hashing.
- Access tokens and API credentials must never be stored as plain text.
- WhatsApp webhook requests must be verified.
- Every request must establish school and user context.
- RBAC must be enforced in Rust.
- AI-generated commands must pass normal authorization and validation.
- SQL must use parameterized queries.
- Destructive operations should support confirmation.
- Important mutations should produce audit records.
- Database transactions should be used for operations that must succeed or fail atomically.
- Secrets must be provided through secure environment/secrets management.
- External webhook processing should support idempotency.

31 Important Design Decisions

31.1 Student vs Admission

An admission application is a process and a potential future student. A student is a persistent identity in the school system.

Therefore:

```
Admission -> Approval -> Student -> Enrollment
```

This prevents admissions from corrupting the permanent student model.

31.2 Student vs Enrollment

A student can exist for many academic years, but enrollment changes each year.

Therefore:

```
Student
|
+-- Enrollment 2026-27 -> Class 8A
|
+-- Enrollment 2027-28 -> Class 9B
```

31.3 AI vs Business Logic

AI should understand language. Rust should enforce rules.

```
AI:
"What the user means"

Rust:
"Whether the user is allowed to do it
and how it is safely executed"
```

31.4 WhatsApp vs Web

WhatsApp is optimized for operational convenience. The web interface is optimized for visual complexity and administration.

Neither should have separate business logic.

32 Initial WhatsApp Command Set

The first conversational MVP can focus on a small, high-value command set.

Command Category	Example
Attendance	Mark Rahul absent in 8A
Attendance	Mark Rahul and Arjun absent today
Attendance	Take attendance for 8A
Attendance correction	Correct Rahul's attendance
Attendance report	Show attendance for 8A this month
Student lookup	Show Rahul's student details
Class lookup	Show students in 8A
Low attendance	Show students below 75% attendance
Routine information	Show today's timetable

The command set should expand only after authentication, permission, validation, and audit mechanisms are proven.

33 Future Modules

The architecture is intended to expand into:

- Advanced admissions.
- Full timetable management.

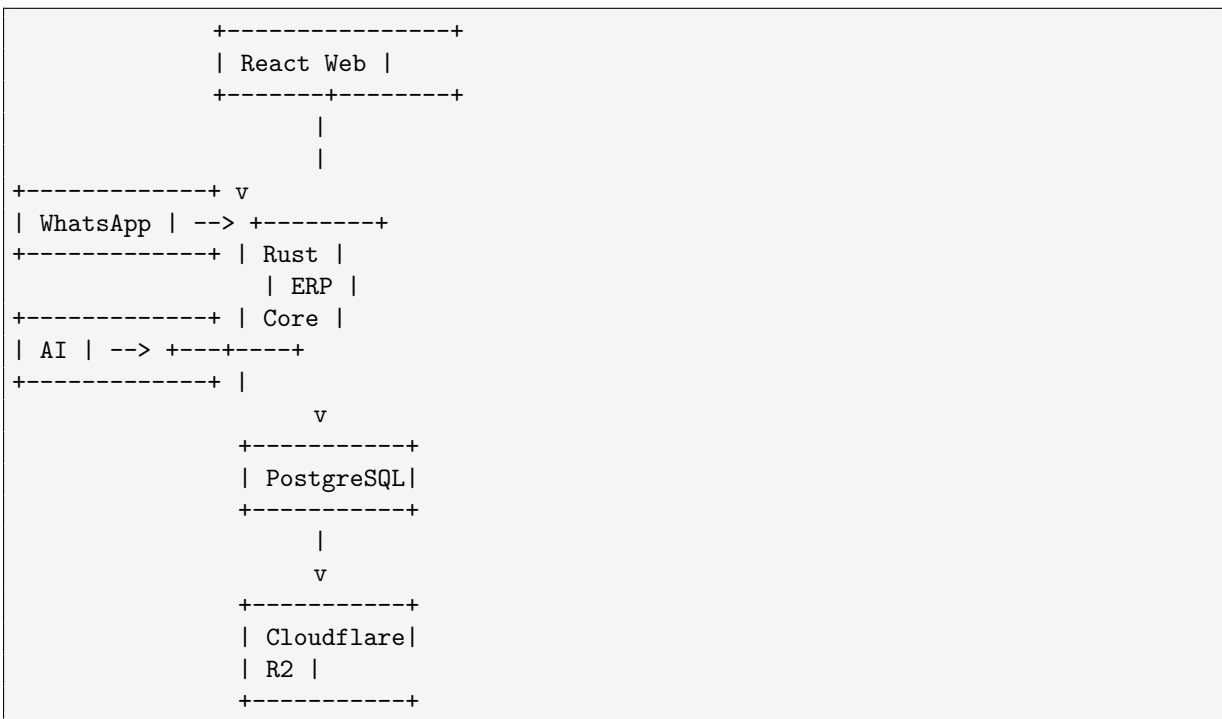
- Teacher substitution management.
- Examinations and report cards.
- Fees and payment integrations.
- Accounting.
- Transport.
- Library.
- Hostel.
- HR and payroll.
- Parent communication.
- Advanced analytics.
- AI-assisted reporting.
- Automated notifications.

These modules should reuse the same identity, authorization, school, academic-year, audit, notification, file, and integration foundations.

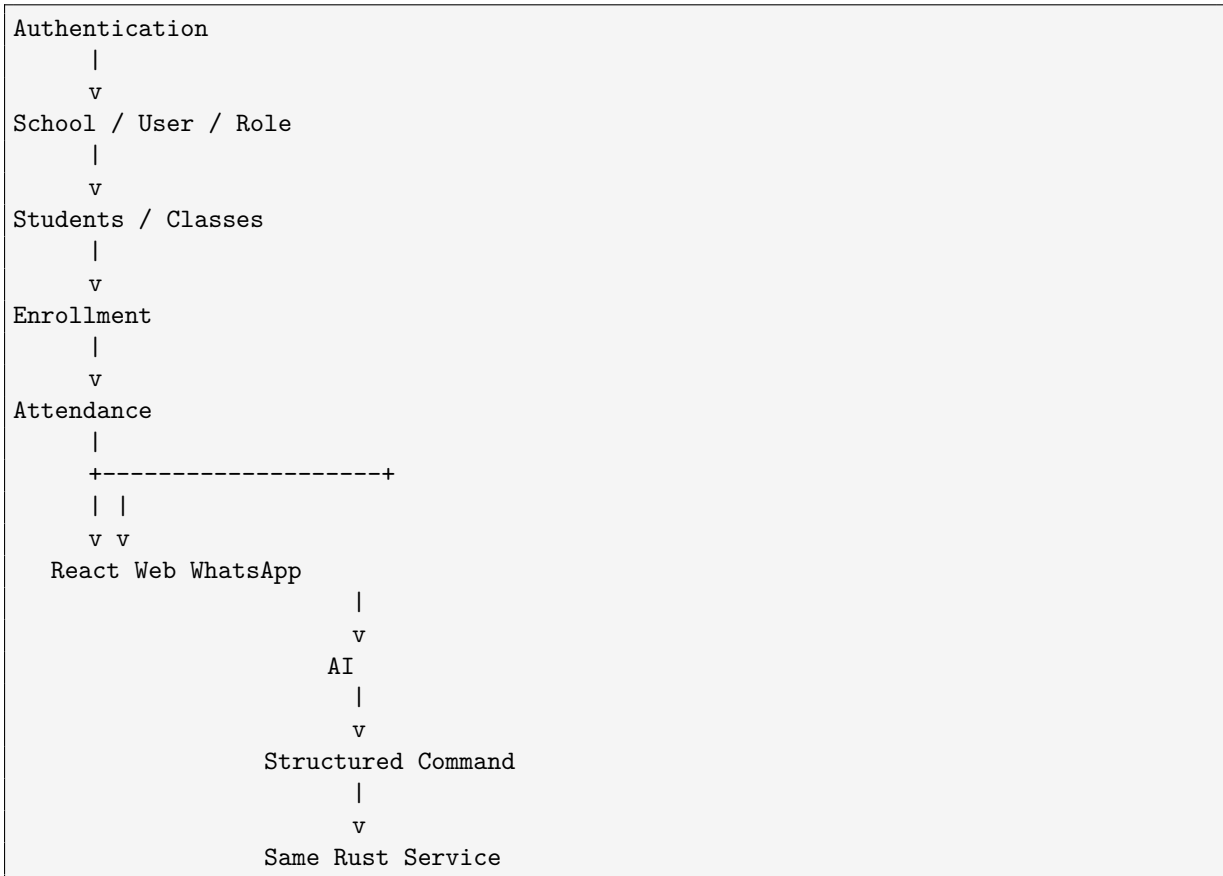
34 Conclusion

The ERP should be built as a single centralized school-management platform with multiple interfaces rather than as separate WhatsApp and web products.

The fundamental architecture is:



The first production-quality vertical slice should be:



This vertical slice establishes the core pattern that every future ERP module will follow.